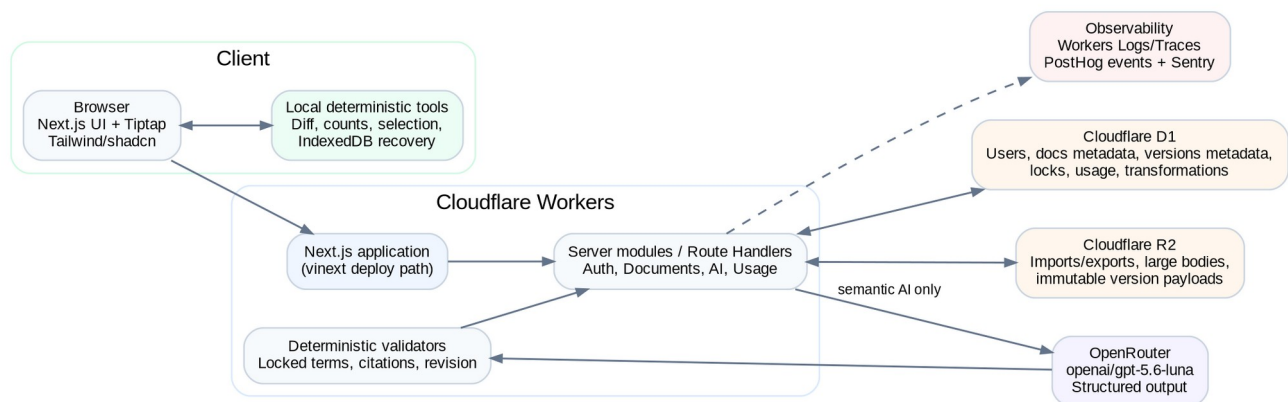


TECHNICAL ARCHITECTURE & BUILD BLUEPRINT

AI Writing Workspace - Engineering Supplement to the Master PRD

Indonesia-first | Bilingual ID/EN | MVP architecture | 12 September 2026



Locked core stack

Next.js + TypeScript + Tailwind CSS on Cloudflare Workers; Cloudflare D1 for relational data; Cloudflare R2 for object storage; OpenRouter for AI with openai/gpt-5.6-luna. Note: Cloudflare does not have a D2 object-storage product; this blueprint interprets the requested "D2" as R2.

This document is intentionally implementation-focused. It does not replace the product PRD; it closes the missing engineering-planning layer: exact stack, service boundaries, data/storage design, API flow, security, deployment, QA, and build sequence.

0. Scope, Cross-check Result, and Locked Technical Decisions

I cross-checked this supplement against the 42-page master PRD. Every MVP capability that requires an engineering path is mapped in the coverage matrix near the end of this document. No product behavior is intentionally changed here. Where the master PRD left technical choices open, this document recommends a default suitable for an indie/SaaS build on Cloudflare.

Decision	Status	Technical direction
Application framework	LOCKED	Next.js + TypeScript. Deploy to Cloudflare Workers using Cloudflare's currently recommended vinext path; keep OpenNext as compatibility fallback only if a blocking vinext gap appears.
Styling/UI	LOCKED	Tailwind CSS. Recommended component primitives: shadcn/ui + Radix; icons: Lucide.
Primary data store	LOCKED	Cloudflare D1 (SQLite semantics) + Drizzle ORM/Drizzle Kit.
Object storage	LOCKED / corrected naming	Cloudflare R2, interpreted from the requested "D2". Use private buckets.
AI gateway	LOCKED	OpenRouter server-side only.
Primary model	LOCKED	openai/gpt-5.6-luna.
Rich-text editor	RECOMMENDED	Tiptap (ProseMirror-based), client-side editor initialization.
Authentication	RECOMMENDED	Better Auth + Drizzle adapter + D1. Add Google OAuth; optional email/password or magic link.
Transactional email	RECOMMENDED	Resend, only if email verification/magic link/password reset is used.
Product analytics	RECOMMENDED	PostHog event analytics, with editor text/session replay capture disabled.
Backend observability	RECOMMENDED	Cloudflare Workers Logs/Traces; Sentry optional for frontend/error aggregation with PII scrubbing.
Rate limit	RECOMMENDED	Cloudflare Workers Rate Limiting API for abuse protection; D1 UsageLedger for accurate plan/quota accounting.
Async jobs	RECOMMENDED	Cloudflare Queues only for asynchronous work such as import/export or offline evaluation. Do not put interactive rewrite in a queue.
Collaboration	LATER	Durable Objects only when real-time multi-user collaboration is introduced.

Architecture principle

Start as a modular monolith, not microservices. One Next.js codebase, one primary Worker deployment, one D1 database, and one R2 bucket set are enough for MVP. Service boundaries live in code first; split into separate Workers only when load or operational evidence requires it.

0.1 What this supplement adds that the master PRD did not lock

- Concrete deployment path for Next.js on Cloudflare in 2026.
- Exact recommended libraries for editor, database access, auth, validation, forms, testing, analytics, and monitoring.
- D1/R2 responsibility split and document-size strategy.
- Detailed request lifecycle for AI preview/apply, versioning, autosave, and concurrency.
- Rate limiting, idempotency, quotas, privacy controls, and provider routing around OpenRouter.
- Repository/module structure, CI/CD, staging/production separation, and rollout order.
- Database tables/indexes and detailed API contracts.
- Feature-to-component coverage matrix to make omissions visible before build.

1. Final Recommended Technology Stack

Layer	Recommendation	Why / rule
Web framework	Next.js 16-compatible codebase	App Router, React Server Components where useful, Route Handlers for explicit APIs.
Cloudflare adapter	vinext	Cloudflare recommends vinext for new Next.js apps on Workers as of Aug 2026. Keep code compatible with next dev; use OpenNext only if a concrete compatibility blocker exists.
Language	TypeScript strict mode	Shared types across UI, DB, API, prompt contracts.
Styling	Tailwind CSS	Fast, consistent design system and responsive states.
UI primitives	shadcn/ui + Radix + Lucide	Accessible primitives without locking into a heavy design framework.
Rich-text editor	Tiptap	ProseMirror document model, selection APIs, custom marks for locked terms, history support.
Database	Cloudflare D1	Serverless SQL with SQLite semantics and Worker bindings.
ORM/migrations	Drizzle ORM + Drizzle Kit	Strong TypeScript fit and first-class D1 integration.
Object storage	Cloudflare R2	Private unstructured storage for files, snapshots, large document bodies.
Authentication	Better Auth	Self-hosted app auth with Drizzle adapter; avoids separate auth database vendor.
AI provider layer	Internal AI adapter -> OpenRouter	Prevents editor/business logic from being coupled to one gateway/model.
Primary AI model	openai/gpt-5.6-luna	Fast/cost-efficient model selected for MVP; supports structured outputs via OpenRouter.
Schema validation	Zod	Shared request/response validation and structured-output validation.
Forms	React Hook Form + Zod	Structured Customize UX with clear validation.
Diff	Intl.Segmenter + diff/jsdiff	Word/sentence-aware deterministic diff; no LLM needed.
Client recovery	IndexedDB via small wrapper (e.g. idb-keyval)	Protect unsaved editor state during network/save failure.
Product analytics	PostHog events only	Funnels/feature adoption; explicitly do not capture document text.
Infra observability	Cloudflare Logs/Traces	Native backend visibility.
Error tracking	Sentry optional	Useful for browser/runtime exceptions; scrub editor/AI payloads.
Testing	Vitest + React Testing Library + Playwright	Unit/component/E2E coverage.
CI/CD	GitHub Actions + Wrangler	Quality gates, migration checks, staging, production deploy.
Package manager	pnpm	Fast deterministic installs and good workspace support if a job Worker is added later.

Do not add Redis, a vector database, Kubernetes, a separate Node backend, or microservices in MVP. None is required by the approved product scope.

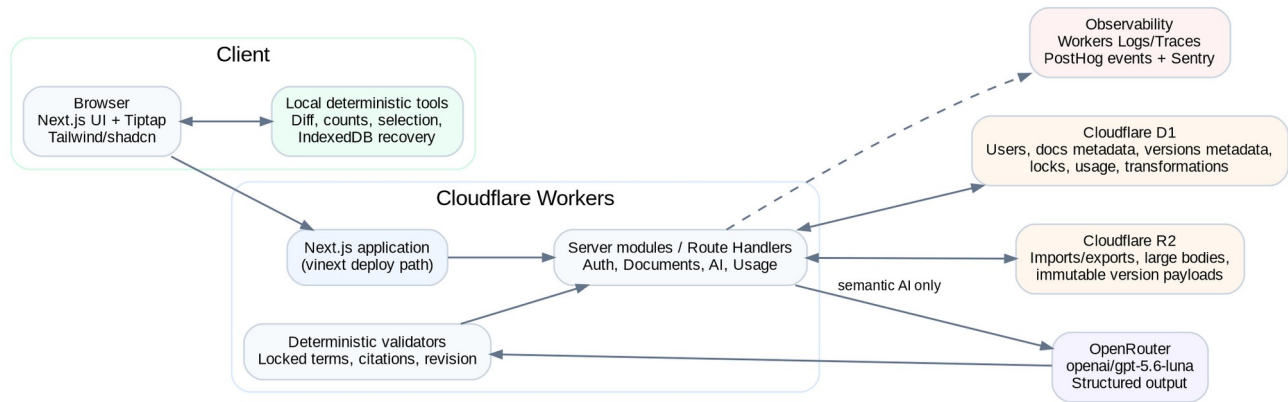
1.1 Why vinext is the default deployment path

Cloudflare currently recommends vinext as the default path for Next.js applications on Workers. For a new project, adopt this recommendation instead of starting with the older OpenNext adapter. Keep an explicit compatibility checklist for the Next.js APIs used by the app; if a required feature has a vinext gap, OpenNext remains a documented fallback rather than a second runtime maintained in parallel.

1.2 Runtime compatibility rule

- Prefer Web Platform APIs (fetch, Request, Response, crypto, streams) over Node-only APIs.
- Every dependency must be verified against Cloudflare Workers before adoption.
- No filesystem assumptions at runtime. Persistent files belong in R2.
- No server process memory may be treated as persistent state.

2. High-Level System Architecture



The browser handles the editor and cheap deterministic interactions. Cloudflare Workers hosts the Next.js server surface and all sensitive logic. D1 stores relational state. R2 stores large/unstructured objects. Only server-side AI modules may call OpenRouter. The browser never receives the OpenRouter key or system prompts.

2.1 Logical modules inside the modular monolith

Module	Responsibility	Calls AI?
Auth	sessions, identity, ownership checks	No
Documents	create/load/autosave/delete document metadata and working copy	No
Versions	immutable checkpoints, restore, history	No
Editor deterministic engine	counts, segmentation, diff, pure formatting	No
Protected content	term locks, citation extraction, post-generation validation	No
AI Orchestrator	prompt selection, context building, OpenRouter call, structured-output parse	Yes
Transformations	persist preview metadata, apply/discard state, idempotency	Mixed
Usage & quotas	plan allowance, token/cost metadata, enforcement	No
Analytics	event metadata only; no raw doc content	No
Import/export	file handling and conversion, V1.1	Usually no

2.2 Browser responsibilities

- Tiptap editing and selection UX.
- Formatting commands, undo/redo, local word/character/read-time metrics.
- Inline floating toolbar and panel state.
- Local diff rendering from server-provided or locally computed texts.
- Debounced autosave requests and local IndexedDB recovery buffer.
- Never compile or expose system prompts; never call OpenRouter directly.

2.3 Server responsibilities

- Authenticate every document/AI request and enforce document ownership.
- Normalize Custom Writing Request fields and resolve server-trusted locked terms/citations.
- Enforce rate limits and accurate quota checks before AI spend.
- Compile versioned prompt policy and call OpenRouter.
- Validate AI result deterministically before preview is marked safe.
- Apply preview only if document base revision is still valid.
- Write version snapshots and usage ledger atomically where possible.

3. Repository and Code Organization

Use one repository and one deployable Next.js app for MVP. Organize around product domains rather than horizontal “utils everywhere”. A future queue consumer can live in the same repository without forcing a monorepo on day one.

```
src/
  app/
    (marketing)/
    (auth)/
    app/
    documents/[id]/
    api/
      auth/[...all]/
      documents/
      ai/
      usage/
  components/
    ui/
    editor/
  features/
    quick-start/
    writing-modes/
    customize/
    humanizer/
    compare/
    version-history/
    analytics/
  server/
    auth/
    documents/
    versions/
    ai/
      adapter/
      prompts/
      schemas/
      validators/
    usage/
    storage/
  db/
    schema/
    migrations/
    client.ts
  lib/
    text/
    diff/
    language/
    errors/
  types/
  workers/
    jobs/                # only when Queues are introduced
  tests/
    unit/
    integration/
    e2e/
    ai-evals/
```

3.1 Dependency direction

- UI may depend on shared types and feature services, but not directly on D1/OpenRouter.
- Route Handlers call server-domain services; domain services call DB/storage/AI adapters.
- AI prompt modules may import schemas and policy helpers, but must not import UI code.
- Database schema is not used as the client contract directly; API schemas remain explicit.

3.2 Prompt files

Prompt text stays in server-only versioned files, for example `src/server/ai/prompts/P03_HUMANIZER/v1.ts`. Each prompt export must include `promptId`, version, supported controls, output schema, and tests. Do not store hidden prompt text in the browser bundle. D1 records the prompt ID/version used for each AI transformation for traceability.

4. Frontend and Workspace Architecture

The approved UI is a document-centric workspace, not a permanent two-box paraphraser. The architecture should preserve that product decision.

4.1 Page composition

Area	Implementation	Important rule
Left navigation	Server-loaded document metadata + client navigation	No document content in navigation payload.
Top bar	React component + autosave state	Compare/History do not call AI.
Editor canvas	Tiptap client component	Canonical editor state is ProseMirror JSON, not arbitrary HTML.
Right AI panel	Client state + explicit API calls	Generation returns preview; never mutates editor automatically.
Inline toolbar	Tiptap selection extension	Send smallest selected scope to AI.
Bottom version timeline	Metadata from D1	Load full historical body only when selected/compared.

4.2 Tiptap recommendation

Tiptap is recommended because the product needs reliable selection handling, rich-text JSON persistence, inline marks, custom nodes, and extension points. In Next.js, initialize it client-side (for example `immediatelyRender: false`) to avoid SSR hydration mismatch. Use only required extensions; do not load collaboration extensions in MVP.

4.3 Custom editor marks / metadata

- `lockedTerm` mark: visually indicates user-protected text and stores lock ID.
- `citationProtected` mark: optional UI treatment for auto-detected citations in Academic mode.
- AI suggestion preview must not be inserted into the canonical editor document until Apply.
- Keep “current AI preview” outside Tiptap state so Discard is trivial.

4.4 Client state strategy

- Use Tiptap for document/editor state.
- Use React state/context for panels, selected mode, and transient UI.
- Use TanStack Query only if client-side server caching materially simplifies document/history requests; it is recommended but not mandatory.
- Do not introduce Zustand/Redux on day one unless state coupling becomes demonstrably painful.
- React Hook Form + Zod owns the structured “Sesuaikan” form.

5. Deterministic Text Engine: What Must Not Use AI

A major cost and latency advantage comes from aggressively keeping frequent text utilities deterministic.

Capability	Recommended implementation	AI call
Word count	Intl.Segmenter or safe Unicode tokenizer	No
Sentence count	Intl.Segmenter sentence segmentation with fallback heuristics	No
Character count	JavaScript string/grapheme logic	No
Reading time	word count / configured reading speed	No
Diff	word/sentence token arrays + jsdiff/diffArrays	No
Change percentage	derived from diff operations	No
Repeated terms	normalized frequency map / n-gram heuristic	No
Term lock	Tiptap mark + D1 LockedTerm record	No
Citation recognition	regex/rules for supported patterns	No
Protected-span validation	exact/normalized comparison	No
Pure bullet formatting	editor list command if semantics already separated	No
Autosave/history	D1/R2 application logic	No

5.1 Indonesian text handling

Use Unicode-aware segmentation and keep user override available. Automatic language detection is convenience only; do not let it silently override an explicit user language choice. Maintain Indonesian-specific QA fixtures for punctuation, abbreviations, citations, acronyms, names, and mixed English terminology.

5.2 Diff granularity

- Default compare view: word-level diff for readability.
- Use sentence-level anchors first for long paragraphs, then word-level diff inside changed sentences if performance becomes an issue.
- Do not diff raw HTML. Diff normalized plain text or structured text segments and map results back to display spans.

6. D1 Database Architecture

Cloudflare D1 is the relational system of record for identity linkage, document metadata, version metadata, lock metadata, AI transformation audit metadata, and accurate usage accounting. D1 uses SQLite semantics and should be accessed through a typed server layer; Drizzle is the recommended ORM/migration tool.

6.1 Core tables

Table	Core fields	Purpose
users	id, email, name, language_preference, primary_use_case, plan, created_at	Product user profile; auth tables are managed by Better Auth.
documents	id, owner_id, title, language, current_mode, revision, body_storage, body_key/body_inline, excerpt, stats, timestamps	Current document pointer/state and metadata.
document_versions	id, document_id, parent_version_id, label, source_type, mode, scope_type, snapshot_r2_key, snapshot_hash, created_at	Immutable timeline checkpoint metadata.
ai_transformations	id, user_id, document_id, base_revision, idempotency_key, prompt_id, prompt_version, model, scope_type, normalized_controls, status, preview_storage/key, usage metadata, timestamps	Traceable preview/apply lifecycle and cost metadata.
locked_terms	id, document_id, exact_text, normalized_text, source, created_at	Server-authoritative protected terms.
usage_ledger	id, user_id, transformation_id, model, input_tokens, output_tokens, cost_usd_if_available, created_at	Accurate plan/quota and cost accounting.
document_events	optional: document_id, type, metadata, created_at	Only if product needs audit events not covered by versions/transformations.

6.2 Required indexes

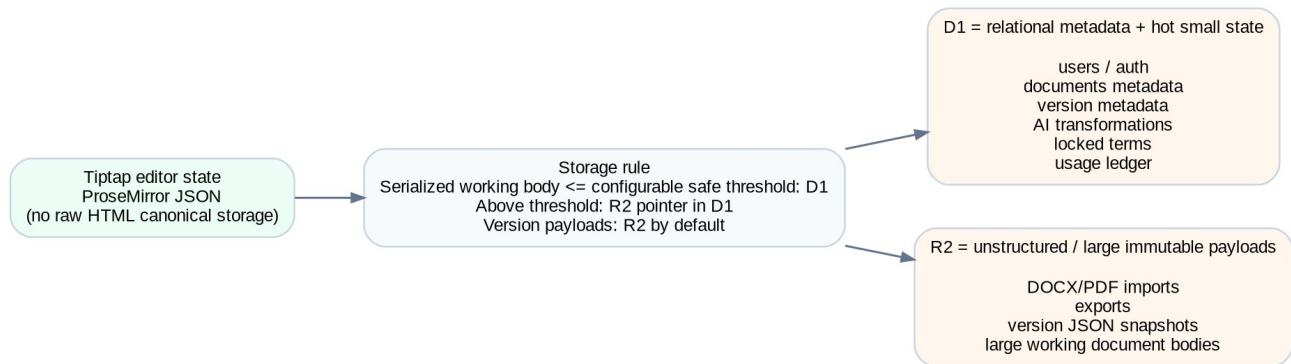
- users(email) UNIQUE.
- documents(owner_id, updated_at DESC) for Recent Documents.
- documents(owner_id, deleted_at) if soft-delete/trash is added.
- document_versions(document_id, created_at DESC).
- ai_transformations(document_id, created_at DESC).
- ai_transformations(user_id, created_at DESC) for usage/debug queries.
- ai_transformations(idempotency_key) UNIQUE.
- locked_terms(document_id).
- usage_ledger(user_id, created_at DESC).

6.3 D1-specific design constraints

- Keep queries indexed; D1 billing is influenced by rows read/written, so avoid full table scans.
- D1 has a 2 MB maximum row/string/BLOB size. Never assume an unlimited rich-text body can live in one row.
- Use D1 batch operations for multi-statement atomic updates such as Apply AI result + version metadata + transformation status when all payload references are already stored.
- Use revision-based optimistic concurrency; do not rely on in-memory locks.

7. R2 Object Storage Architecture

R2 is the correct Cloudflare product for object storage. It should be private by default. The application serves or signs access only after authorization.



7.1 What belongs in R2

- Imported DOCX/PDF source files (V1.1).
- Exported DOCX/PDF files (V1.1), preferably with expiration/cleanup policy if generated frequently.
- Immutable document version payloads (recommended default).
- Large working document bodies that exceed the configured D1 inline threshold.
- Optional AI preview payloads only when too large for D1; small previews can remain in D1 application storage.

7.2 Object key convention

```
documents/{ownerId}/{documentId}/versions/{versionId}.json.gz  
working/{ownerId}/{documentId}/current.json.gz  
imports/{ownerId}/{documentId}/{fileId}/{originalName}  
exports/{ownerId}/{documentId}/{exportId}.{ext}
```

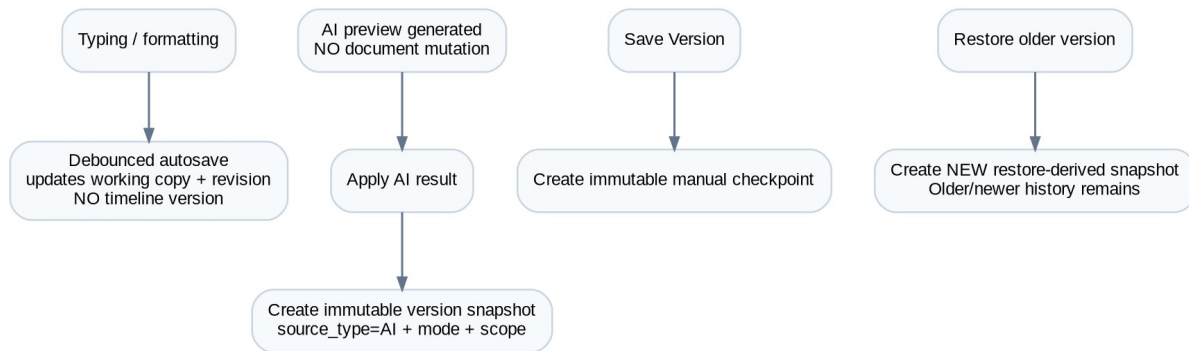
7.3 Immutability rule

Version objects use unique version IDs and are never overwritten. Restoring a historical version creates a new version object. This mirrors the user-facing “Git-like safety without Git terminology” decision.

7.4 Direct upload rule

For small MVP text-only operation, no R2 upload is needed. When V1.1 file upload is introduced, prefer authenticated presigned/direct upload or Worker-mediated upload with strict size/type validation. Never expose a public write bucket.

8. Document Body, Autosave, and Version Storage Strategy



8.1 Canonical document representation

Store editor content as Tiptap/ProseMirror JSON. Derive plain text for AI and analytics at request time or maintain a small excerpt/stat cache. Do not use arbitrary HTML as the canonical database representation because sanitization, diffing, and structural migrations become harder.

8.2 Working-body storage rule

- If serialized JSON is below a conservative configurable threshold (recommended starting point: 512 KB), it may be stored inline in D1 for fast autosave.
- If it exceeds the threshold, switch the document to R2-backed working-body storage; D1 holds storage mode, R2 key, hash, revision, excerpt, and metrics.
- The threshold is a safety margin beneath D1's 2 MB row limit, not a product-facing promise.
- Immutable version snapshot payloads should be written to R2 by default so version history does not inflate relational rows.

8.3 Autosave algorithm

1. Editor changes mark the document dirty immediately and write a recovery copy to IndexedDB.
2. After roughly 1.5-2.5 seconds of user idle (configurable), send PATCH /autosave with expected_revision and content hash.
3. Server authorizes ownership and performs optimistic-concurrency update.
4. On success, server increments revision; client clears/updates the IndexedDB recovery marker.
5. On 409 revision conflict, do not overwrite the server silently. Show a "newer copy exists" recovery flow.
6. Before an AI request, flush any pending autosave or explicitly bind the AI request to the current unsaved text + base revision so the preview is deterministic.

8.4 Meaningful versions vs autosave

Autosave protects work but does not create timeline noise. AI Apply, manual Save Version, and Restore create immutable timeline checkpoints. This preserves the master PRD behavior exactly.

9. Authentication and Authorization

Recommendation: Better Auth with the Drizzle adapter on D1. This keeps session/user data inside the same Cloudflare relational platform and avoids adding a separate auth database SaaS. Better Auth supports standard Request/Response server runtimes and a Drizzle SQLite adapter.

9.1 MVP sign-in

- Google OAuth is recommended because the primary audience is students/professionals and it minimizes password friction.
- Optionally add email/password or magic link. If magic link/password reset is enabled, use Resend for transactional email.
- Use httpOnly, Secure, SameSite cookies according to Better Auth recommendations.
- Every document query must include owner_id authorization; never rely on client-provided ownership.

9.2 Authorization model

Resource	MVP policy
Document	single owner only
Version	inherits document owner
Locked term	inherits document owner
AI transformation	requesting user must own document or be operating on an unsaved Quick Start draft
R2 object	never directly public; access only after server authorization or short-lived signed URL
Settings/usage	user-scoped

9.3 Future collaboration boundary

Do not pre-build organizations, roles, comments, or real-time ACL complexity into MVP. When team/university collaboration is validated, introduce membership tables and Durable Objects for coordinated real-time document state.

10. AI Architecture: OpenRouter + GPT-5.6 Luna

All AI calls originate server-side through an internal adapter. The initial model is OpenRouter slug `openai/gpt-5.6-luna`. The adapter makes model/gateway changes possible later without rewriting UI or domain services.

10.1 AI adapter interface

```
transform(request: NormalizedTransformRequest): Promise<TransformResult>
alternatives(request: AlternativesRequest): Promise<AlternativesResult>
analyzeQuality(request: QualityRequest): Promise<QualityResult>
repair(request: RepairRequest): Promise<RepairResult>
```

10.2 Structured output

Use OpenRouter structured outputs / JSON schema when supported by the selected endpoint. The response schema should contain `transformed_text`, `change_categories`, `warnings`, and only task-specific optional fields. Validate the parsed result again with Zod server-side; model-side schema enforcement is not a substitute for application validation.

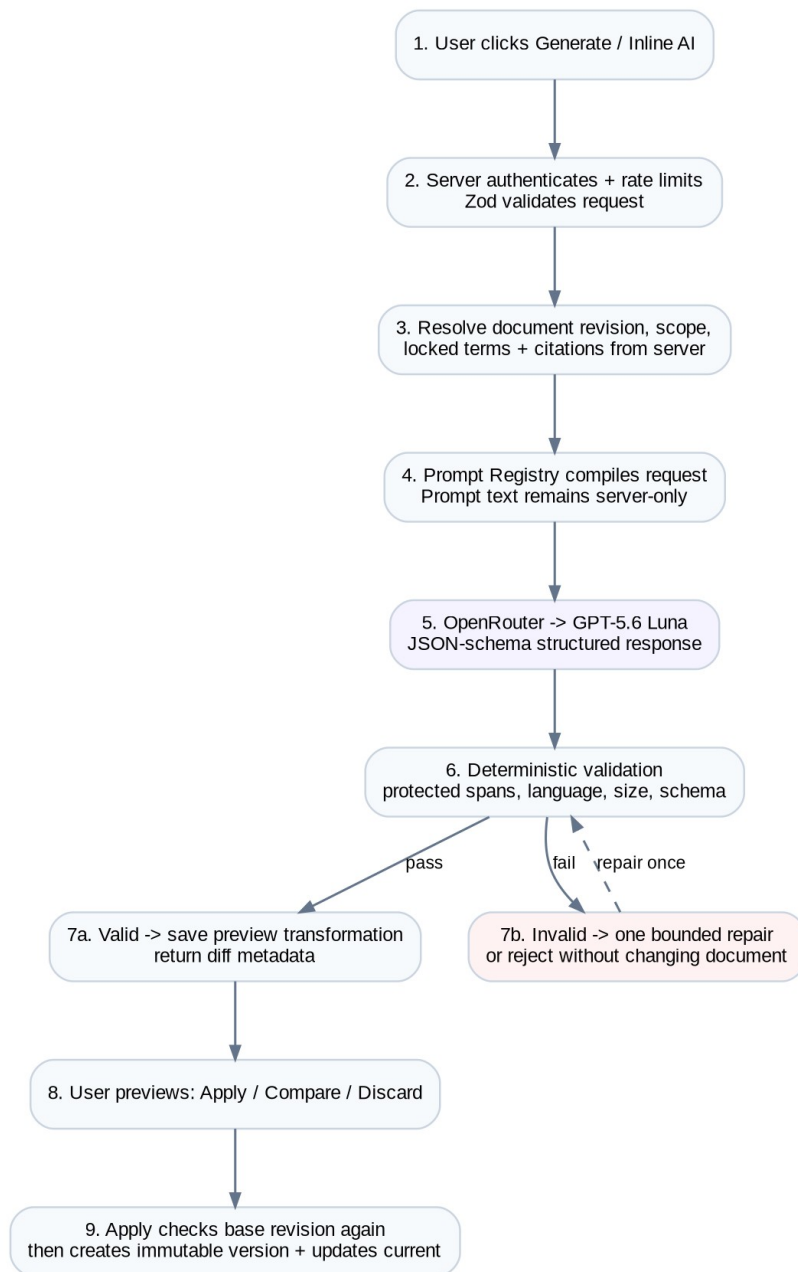
10.3 Provider routing and reliability

- Allow OpenRouter to fail over across healthy providers for the same GPT-5.6 Luna model when policy permits.
- Do not silently fall back to a different model family in MVP; output consistency matters for prompt evaluation. A cross-model fallback can be introduced only after evaluation.
- Network/5xx retries should be bounded with exponential backoff. Application-level protected-span repair is a separate single retry.
- Attach request IDs so logs can correlate app request -> OpenRouter request -> transformation record without logging raw text.

10.4 Privacy policy for OpenRouter

- Keep OpenRouter input/output logging disabled unless explicitly required for a controlled debugging environment.
- Use OpenRouter provider privacy controls, including `data_collection` restrictions and ZDR where an eligible endpoint can satisfy the selected model.
- If strict ZDR cannot serve the selected model/provider, do not silently relax privacy. Make the product policy explicit and fail closed for any privacy tier that promises ZDR.
- Never forward document text into product analytics, Sentry breadcrumbs, or generic console logs.

11. End-to-End AI Transformation Sequence



11.1 Why Preview and Apply are separate endpoints

The preview may be generated against document revision N, while the user keeps typing. Apply must therefore verify that the source text/revision is still compatible. This prevents an old preview from overwriting newer manual edits.

11.2 Request contract

Field	Rule
idempotency_key	Client-generated UUID; unique in D1. Same key returns the existing transformation instead of charging AI twice.
document_id	Optional for anonymous/Quick Start drafts; required inside saved workspace.
base_revision	Required for saved documents.
scope_type	selection sentence paragraph document.
source_text	Only requested scope, not full history.
context_before/context_after	Minimal neighboring context if needed.
mode	standard academic humanize professional creative simplify.
mode_settings	strength/context/preservation settings relevant to selected mode.
custom_request	normalized format, length, audience, focus, short extra instruction.
language	auto result resolved/overridden to id or en.

11.3 Apply contract

- Apply accepts transformation_id, document_id, expected_revision, and target anchor/range metadata.
- Server loads the stored validated preview; client does not get to replace the preview with arbitrary text under the same transformation ID.
- If revision changed and the exact source span no longer matches, return conflict and ask the user to regenerate/rebase the preview.
- On success: write immutable version payload, insert version metadata, update current document and revision, mark transformation applied, and write usage linkage.

12. Prompt Registry and System Prompt Implementation

The master PRD intentionally leaves prompt text TBD. Engineering should make that research swappable and testable, not hard-coded across route files.

12.1 Registry shape

Prompt ID	Purpose	Implementation rule
P01_STANDARD_REWRITE	neutral paraphrase	server-only file + version
P02_ACADEMIC	academic rewrite	server-only file + version
P03_HUMANIZER	context-aware humanization	server-only file + version
P04_PROFESSIONAL	professional rewrite	server-only file + version
P05_CREATIVE	creative rewrite	server-only file + version
P06_SIMPLIFY	simplification	server-only file + version
P07_INLINE_ALTERNATIVES	3-5 scoped alternatives	server-only file + version
P08_CUSTOM_TRANSFORM	structured custom request	server-only file + version
P09_QUALITY_EVAL	qualitative analysis	server-only file + version
P10_REPAIR	repair protected-span violation	server-only file + version
BASE_POLICY	shared rules	composed with capability prompt; does not replace it

12.2 Prompt versioning rule

- Each prompt has semantic or integer version (e.g. P03 v1, v2).
- Every ai_transformations row stores prompt_id + prompt_version + model slug.
- Prompt changes require the golden AI evaluation suite before production.
- Never edit an old version in place if it has already produced production data; add a new version.
- Feature flags may select a prompt version for A/B evaluation, but raw prompt text stays server-side.

13. Humanizer Engineering Pipeline

Humanizer is a hero feature and should receive stricter validation than a generic rewrite.

7. Capture scope, context, language, Humanizer context, strength, preservation mode, locked terms, detected citations, and critical numeric spans.
8. Compute local pre-metrics: word/sentence count, repeated phrase candidates, diff baseline.
9. Compile P03_HUMANIZER + BASE_POLICY. System prompt text remains TBD by prompt research.
10. Call GPT-5.6 Luna once with structured output.
11. Validate schema, locked terms, citations, required numeric spans, and max-change guardrails appropriate to strength.
12. If a protected element changed, run at most one P10_REPAIR call; otherwise reject the preview.
13. Compute deterministic diff/change %. Return preview + concise change categories.
14. Apply only after explicit user action and base-revision check.

13.1 Humanizer must not

- Insert typos, bad grammar, random punctuation, fake anecdotes, or unsupported facts.
- Downgrade Academic context into casual writing unless the user selects another context.
- Optimize or market itself as an AI-detector bypass tool.
- Change locked terms/citations silently.
- Resend the whole document when the user selected a small passage.

13.2 Optional quality evaluation

P09_QUALITY_EVAL is a separate on-demand call. Do not automatically call it after every Humanizer request; that would nearly double AI traffic without proving corresponding user value.

14. Protected Terms, Citations, and Validation

Prompt instructions alone are not enough for high-trust academic protection. The server must validate.

14.1 Locked terms

- User selects exact text -> "Kunci Istilah".
- Client adds editor mark; server creates locked_terms record.
- AI request sends only locks relevant to the selected scope/context.
- Post-generation validator checks exact presence and/or position policy.
- If user explicitly requests changing a locked term, the UI requires Unlock first.

14.2 Citation detection

Support common patterns incrementally with deterministic regex/rules, for example parenthetical author-year and narrative author-year forms. Because citation syntax is varied, manual lock remains authoritative. The validator must not pretend unsupported styles are fully protected.

14.3 Numbers and factual spans

For Humanizer/Academic modes, capture critical numeric tokens and named entities as validation hints. Do not automatically hard-lock every number if the user asked to reformat a range or unit; validation should distinguish "must preserve factual value" from "must preserve exact string".

15. Rate Limiting, Quotas, Idempotency, and Cost Control

15.1 Two different mechanisms

Mechanism	Tool	Purpose
Abuse/request rate limit	Cloudflare Workers Rate Limiting API	Protect endpoints from bursts/automation. Approximate/per-location; not billing truth.
Accurate plan quota	D1 usage ledger	Count successful AI usage against plan in a durable, auditable way.

15.2 AI request flow before spend

15. Authenticate user.
16. Validate payload size and plan limits.
17. Check approximate abuse rate limiter.
18. Check D1 quota/allowance.
19. Check idempotency key; if existing completed transformation exists, return it.
20. Only then call OpenRouter.

15.3 Cost controls

- Smallest-scope requests by default.
- Batch 3-5 inline alternatives in one call.
- No AI on typing, formatting, diff, history, or local analytics.
- No automatic P09 quality evaluation after every rewrite.
- Cap additional_instruction length and total user scope by plan.
- Record OpenRouter token/usage metadata where available, but keep an app-side usage ledger independent from provider dashboard.
- Use model/provider prompt caching only after privacy and actual savings are verified; it is not required for MVP.

16. Cloudflare Queues and Durable Objects: When to Use Them

16.1 Cloudflare Queues - recommended later, not in interactive AI path

Queues provide guaranteed asynchronous delivery and are useful when the user does not need the result inside the current HTTP request.

- V1.1 DOCX/PDF export generation if processing becomes slow.
- Large import parsing workflows.
- Offline AI evaluation runs / golden-dataset batches.
- Background cleanup of expired exports/previews.
- Email/event fan-out if needed.

Do not queue normal paraphrase/humanize actions in MVP. The user is waiting for an immediate preview; adding a queue complicates latency and polling without benefit.

16.2 Durable Objects - reserve for collaboration

Durable Objects are appropriate for coordinated, stateful multi-client applications. They become valuable when real-time collaboration, presence, shared cursors, or serialized multi-user edits are introduced. Single-user MVP does not need them.

17. Product Analytics, Usage Analytics, and Observability

17.1 Product analytics

Recommendation: PostHog for named product events/funnels because the master PRD already defines events such as `generation_started`, `result_applied`, `compare_opened`, `inline_action_used`, and `humanizer_applied`. Configure analytics so editor content is never captured.

- Disable or strictly exclude session replay on editor/document routes.
- Do not send selected text, document title if sensitive, prompt text, AI output, or raw `additional_instruction` into analytics.
- Use stable IDs and categorical metadata only: mode, scope type, result applied/discarded, latency band, plan.

17.2 Usage/cost analytics

Use D1 UsageLedger as accounting truth. Optionally mirror aggregate, non-content metrics to Cloudflare Analytics Engine for dashboards such as transformations/hour, p95 latency, rate limited requests, and token cost by mode.

17.3 Infrastructure observability

- Cloudflare Workers Logs for backend errors and request traces.
- Cloudflare Traces for outbound OpenRouter/D1/R2 timing.
- Sentry optional for frontend exceptions and cross-stack issue aggregation; configure PII scrubbing and disable request-body capture on document/AI routes.
- Every request gets a correlation ID; `ai_transformations` stores the app request ID/provider request metadata without raw text in generic logs.

18. Security and Privacy Implementation

Area	Requirement
Secrets	Cloudflare Secrets / Secrets Store; never NEXT_PUBLIC for OpenRouter/auth secrets.
Transport	HTTPS only; secure cookies.
R2	Private buckets; authenticated access or short-lived signed URL.
Editor content	Canonical JSON; sanitize pasted HTML through editor schema; render only whitelisted nodes/marks.
Authorization	owner check on every document/version/storage operation.
Prompt injection	User document is content, not instruction. Server system policy and structured product controls have precedence.
Logging	No raw document text in console logs, analytics, error breadcrumbs, or tracing attributes.
OpenRouter	Server-side key, logging off, provider data policy/ZDR controls where available.
Rate limits	Per user/tenant/route, not primarily IP.
Deletion	Delete D1 metadata and corresponding R2 objects according to product policy; ensure derived exports/previews are included.

18.1 Content Security Policy

Use a restrictive CSP appropriate to the actual dependencies. Avoid unsafe-eval in production. Explicitly allow only required analytics/error endpoints. Treat R2 media/download domains separately.

18.2 File upload security (V1.1)

- Validate extension, MIME type, and magic bytes where feasible.
- Enforce per-plan size limits before upload processing.
- Use randomized object keys; original filename is metadata only.
- Do not execute macros or embedded scripts. DOCX/PDF are parsed as data.
- Scanned PDFs are rejected or routed to a separately approved OCR feature; do not silently fake extraction.

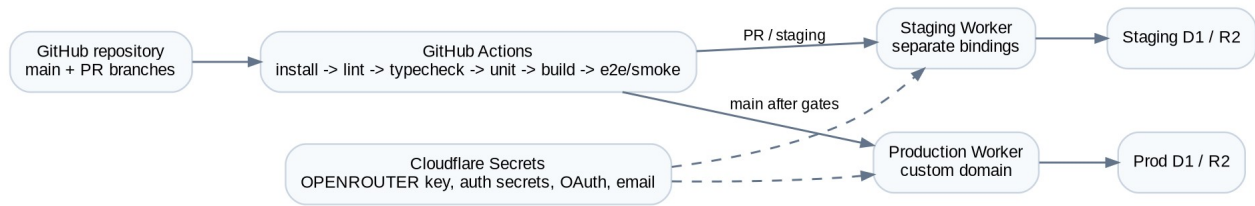
19. Detailed API Surface

Method / route	Purpose	AI?	Key checks
POST /api/documents	Create blank/pasted document	No	auth, size
GET /api/documents	Recent/list documents	No	auth, pagination
GET /api/documents/:id	Load current document	No	ownership
PATCH /api/documents/:id/autosave	Persist working copy	No	ownership, expected_revision
DELETE /api/documents/:id	Delete/archive document	No	ownership
GET /api/documents/:id/versions	Timeline metadata	No	ownership
POST /api/documents/:id/versions	Manual checkpoint	No	ownership, current revision
POST /api/documents/:id/restore	Restore as new version	No	ownership, version exists
GET/POST/DELETE /api/documents/:id/locks	Manage protected terms	No	ownership, exact span
POST /api/ai/transform	Standard/Academic/Humanize/etc preview	Yes	auth/quota/rate/idempotency/revision
POST /api/ai/alternatives	Scoped 3-5 alternatives	Yes	small scope only
POST /api/ai/analyze-quality	On-demand heuristic quality analysis	Yes	auth/quota
POST /api/ai/apply	Apply validated preview	No new AI	ownership, transformation valid, revision/anchor match
POST /api/ai/discard	Mark preview discarded	No	ownership
GET /api/usage	Plan usage summary	No	auth
POST /api/import	V1.1 file import	No/async	auth, file validation
POST /api/export	V1.1 export job	No/async	auth, document access

19.1 Error contract

Code	Meaning	UX behavior
400 INVALID_INPUT	Schema/option conflict	Show field-specific error, keep text.
401 UNAUTHENTICATED	Session missing/expired	Prompt sign-in, preserve local draft.
403 FORBIDDEN	Document not owned	Do not leak existence.
409 REVISION_CONFLICT	Document changed since preview/autosave	Offer refresh/compare/regenerate; never overwrite silently.
422 PROTECTED_CONTENT_VIOLATION	AI output broke lock/citation after bounded repair	Keep preview unapplied and explain protected item.
429 RATE_LIMITED	Burst limit exceeded	Retry after cooldown; do not lose text.
429 QUOTA_EXCEEDED	Plan usage exceeded	Show plan/limit message before another AI call.
502 AI_PROVIDER_ERROR	OpenRouter/provider failed	Retry safely with same idempotency key.
503 STORAGE_UNAVAILABLE	D1/R2 temporarily unavailable	Keep IndexedDB recovery; retry autosave.

20. CI/CD, Environments, and Deployment



20.1 Environments

Environment	Resources
local	Wrangler local D1, local R2 emulation/remote dev only when needed, .dev.vars for secrets
staging	separate Worker, D1, R2, Better Auth config, OpenRouter key/guardrail, analytics project or environment tag
production	separate Worker, D1, R2 and secrets; custom domain

20.2 Pull request quality gates

- pnpm install --frozen-lockfile
- TypeScript typecheck (strict).
- ESLint + formatting check.
- Vitest unit/integration suite.
- Build using the actual Cloudflare deployment adapter.
- Playwright smoke tests against preview/staging for core journey.
- Drizzle migration diff/schema check.
- AI golden eval is required when prompt text or AI validation policy changes; it does not have to run on every unrelated UI PR.

20.3 Database migration strategy

- Generate SQL migrations with Drizzle Kit and commit them.
- Review migrations manually; never auto-push schema changes from production runtime.
- Prefer expand-and-contract migrations: add compatible schema first, deploy app, backfill if needed, remove old fields later.
- Back up/verify D1 Time Travel before high-risk migrations. User version history is not a database backup replacement.

21. Testing and Quality Strategy

Test layer	Tool	What it covers
Unit	Vitest	normalizers, diff helpers, citation regex, lock validator, usage calculation, prompt compiler inputs
Component	React Testing Library	Customize form, AI preview card, inline toolbar, validation states
Editor integration	Vitest/RTL + targeted Tiptap tests	selection scope, lock marks, apply behavior, undo
API integration	Vitest against local Worker/D1	auth, revision conflicts, idempotency, version writes
E2E	Playwright	Quick Start -> Generate -> Apply -> edit -> inline action -> Compare -> Restore
AI evaluation	Golden dataset + rubric	Indonesian/English meaning, naturalness, academic fit, protected spans, unsupported claims
Load/cost smoke	scripted requests	latency, D1 row scans, OpenRouter call counts, quota correctness

21.1 Golden AI dataset

Keep the master PRD categories: thesis introduction, literature review, methodology, discussion, professional email/report, marketing copy, mixed technical terms. Store test inputs and constraint metadata in the private repository; do not use real confidential user documents.

21.2 Humanizer release gate

- 0 locked-term violations in accepted regression cases.
- No deliberate typo/error insertion.
- No new unsupported factual claim in the normal Humanizer test set.
- Academic context remains academic.
- Prompt version change must beat or match the previous version on the agreed rubric before rollout.

22. Performance, Latency, and Cost Engineering

22.1 Latency budget mindset

- Editor typing/formatting: entirely client-side and immediate.
- Autosave: asynchronous and non-blocking.
- Diff/history/local metrics: no AI and should feel instant after data is loaded.
- AI request: optimize payload size first; model latency dominates.
- Apply: should be fast because no second AI call is required unless validation already failed during preview.

22.2 D1 efficiency

- Never fetch all document versions with full bodies just to render the timeline; fetch metadata only.
- Index owner_id, document_id, timestamps used by filters.
- Store large historical payloads in R2 so D1 rows remain small.
- Measure rows_read/rows_written during load tests because D1 pricing and performance are query-shape sensitive.

22.3 AI cost metrics

- Cost per successful applied transformation is the primary unit, not cost per raw API call.
- Measure apply/discard rate by mode. A cheap model is still wasteful if users discard outputs.
- Measure percentage of selection-level vs full-document transformations.
- Measure protected-content repair rate; a high rate means prompt/validator quality is poor.
- Model pricing must be configuration/telemetry data, not hard-coded into product logic because OpenRouter pricing can change.

23. Build Sequence: Recommended Engineering Milestones

This order is designed to prove the core editing loop before adding peripheral complexity.

Milestone	Deliverables	Exit criteria
M0 - Foundation	Next.js/TS/Tailwind project, vinext deploy, D1/R2 bindings, Drizzle, Better Auth, CI	Sign in and deploy staging reliably.
M1 - Document core	Tiptap workspace, create/load/autosave, IndexedDB recovery, document revision	No content loss across refresh/save failure test.
M2 - Version/diff deterministic layer	Save Version, immutable R2 snapshots, timeline, compare, counts/read time	All no-AI PRD flows work.
M3 - AI Standard path	OpenRouter adapter, P01 placeholder, structured output, preview/apply/discard, UsageLedger, idempotency	One end-to-end rewrite safely applies as version.
M4 - Modes + Customize	Academic/Humanize/Professional/Creative/Simplify prompt IDs, structured custom request	All mode settings normalize into same transform API.
M5 - Inline intelligence	selection toolbar, alternatives, shorter/clearer/humanize selection	Small selection never sends full document.
M6 - Academic safety	term lock, citation rules, validator, one repair retry	Locked terms never silently change.
M7 - Humanizer quality	context/strength/preservation controls + regression suite	Humanizer release gate passes.
M8 - Analytics/ops	PostHog events, Workers observability, Sentry scrub, rate limits, dashboards	Operational and funnel telemetry without raw text.
M9 - Optional quality analysis	P09 evaluator and analytics drawer	On-demand only; labeled heuristic.
V1.1	DOCX/PDF import/export, queues as needed, templates	Does not destabilize core workspace.

24. Detailed Feature-to-Architecture Coverage Cross-check

This table is the omission check against the approved master PRD. “Covered” means the technical path is explicitly defined in this supplement.

Master PRD capability	Technical mapping	Status
Writing Intent Engine	UI mode chips -> normalized mode -> Prompt Registry -> /api/ai/transform	Covered
Structured Custom Writing Request	React Hook Form + Zod -> normalized controls -> same transform endpoint	Covered
Smart Rewrite Workspace	Next.js + Tiptap + AI panel + explicit preview/apply	Covered
Inline Rewrite / Alternatives	Tiptap selection -> /api/ai/alternatives or transform with selection scope	Covered
Term Lock	Tiptap mark + locked_terms D1 + validator	Covered
Citation Protection	deterministic regex/rules + validator + Academic mode policy	Covered
Diff / Compare	Intl.Segmenter + jsdiff, no AI	Covered
Version Timeline	D1 metadata + immutable R2 snapshots	Covered
Basic Writing Analytics	client deterministic engine	Covered
AI Writing Quality Snapshot	P09 + /api/ai/analyze-quality, on demand	Covered
Humanizer Change Insights	P03 structured change_categories + deterministic diff	Covered
Humanizer context/strength/preservation	mode_settings + P03 + validator	Covered
Original preserved	preview/apply separation + immutable Original/version snapshot	Covered
Autosave without noisy history	working revision only; meaningful snapshot rules	Covered
Quick Start big input	Next route/page; same transform service	Covered
Dashboard recent documents	D1 indexed owner_id/updated_at query	Covered
Language ID/EN/Auto	client selector + normalized request + QA fixtures	Covered
Google Docs-like single canvas	Tiptap main editor; compare is explicit mode	Covered
AI vs non-AI split	deterministic engine vs AI Orchestrator boundaries	Covered
GPT-5.6 Luna	OpenRouter slug openai/gpt-5.6-luna	Covered
System prompts TBD	server Prompt Registry with placeholder/version contract	Covered
DOCX/PDF import V1.1	R2 + validated async parsing path; Queues if needed	Covered as V1.1
Export V1.1	R2 generated artifacts + optional queue job	Covered as V1.1
Plagiarism/AI detector	explicitly not built in MVP	No implementation by design
Collaboration	Durable Objects reserved for later	Deferred by design

Cross-check conclusion

No approved MVP product capability is missing an implementation path after this supplement. Remaining unknowns are intentionally open product decisions (prompt text, pricing/plan limits, final brand, exact billing provider) rather than architecture gaps.

25. Decisions Intentionally Still Open

Decision	Why not locked here	Recommended timing
Exact system prompt text	User will research separately; architecture supports versioned prompts.	Before AI QA freeze.
Pricing and word/token plan limits	Needs actual OpenRouter usage/apply-rate telemetry.	After prototype/staging telemetry.
Billing provider	Product pricing/subscription model not yet decided.	Before paid launch. Keep billing adapter boundary.
Whether to use PostHog Cloud or another analytics vendor	Recommendation is PostHog, but privacy/commercial preference may change.	Before analytics implementation.
Email auth method	Google OAuth may be enough initially; email provider depends on login flow.	Auth implementation milestone.
DOCX/PDF parser libraries	V1.1 and Cloudflare runtime compatibility must be verified with representative files.	V1.1 technical spike.
Cross-model AI fallback	Would affect output consistency/evals.	Only after Luna reliability data.

25.1 Explicitly not open anymore

- Next.js, TypeScript, Tailwind CSS.
- Cloudflare Workers runtime.
- D1 relational database.
- R2 object storage (correcting “D2”).
- OpenRouter gateway and GPT-5.6 Luna primary model.
- Hybrid AI/deterministic architecture.
- Google Docs-like Tiptap workspace direction.
- Preview-before-apply, version safety, diff, locks, and scoped AI calls.

26. Engineering Definition of Done

- Production build runs on Cloudflare Workers using the selected Next.js deployment adapter and has no unsupported Node-only runtime dependency.
- D1 schema and migrations are committed, indexed, and reproducible from a clean environment.
- R2 buckets are private and all object access is authorization-checked.
- Better Auth ownership/security tests pass.
- Every AI capability maps to one documented prompt ID/version and one Zod response schema.
- OpenRouter API key and prompt text are server-only.
- AI transform requests are idempotent and quota-checked before spend.
- No typing, formatting, compare, history, or local analytics action calls AI.
- All AI previews are validated and require explicit Apply.
- Apply refuses stale base revisions rather than overwriting newer work.
- Locked terms are deterministically protected; Academic citation regression tests pass for supported patterns.
- Humanizer regression suite satisfies quality gates and never intentionally degrades grammar.
- Autosave failure has IndexedDB recovery and visible unsaved status.
- Product analytics and error telemetry do not contain raw document text.
- Core Playwright flow passes in staging.
- Observability can trace a failed transform without exposing user content.
- Cost-per-applied-transformation and AI apply/discard metrics are measurable.
- Production and staging use separate D1/R2/secrets.

27. Recommended Initial Environment Configuration

```
# Server secrets (Cloudflare Secrets)
OPENROUTER_API_KEY=...
BETTER_AUTH_SECRET=...
GOOGLE_CLIENT_ID=...
GOOGLE_CLIENT_SECRET=...
RESEND_API_KEY=...      # only if email auth/transactional email is enabled
SENTRY_AUTH_TOKEN=...   # CI only if Sentry source maps are used

# Public non-secret config
NEXT_PUBLIC_APP_URL=...
NEXT_PUBLIC_POSTHOG_KEY=... # optional
NEXT_PUBLIC_POSTHOG_HOST=...

# Cloudflare bindings (wrangler config, not secrets)
DB -> D1 database
STORAGE -> R2 bucket
RATE_LIMITER -> Workers Rate Limiting binding
ANALYTICS -> Analytics Engine dataset (optional)
AI_JOBS -> Cloudflare Queue (V1.1 / optional)
```

Do not hard-code model price, prompt version, or plan limits into multiple components. Centralize runtime configuration in a typed server config module and record effective values with transformations where auditability matters.

28. Source Notes and Current Platform Facts

The recommendations below were cross-checked against current official documentation as of 12 September 2026. These references support platform capability choices; they do not replace implementation testing in the actual repository.

Reference	URL	Used for
Cloudflare Workers - Next.js	https://developers.cloudflare.com/workers/framework-guides/web-apps/nextjs/	Cloudflare currently recommends vinext as the default path for new Next.js applications on Workers; OpenNext remains a compatibility path.
Cloudflare D1 overview	https://developers.cloudflare.com/d1/	D1 is Cloudflare's managed serverless SQL database with SQLite semantics.
Cloudflare D1 limits	https://developers.cloudflare.com/d1/platform/limits/	D1 maximum row/string/BLOB size is 2 MB; database/query limits inform the D1/R2 split.
Cloudflare D1 batch API	https://developers.cloudflare.com/d1/worker-api/d1-database/	D1 batch statements execute as a transaction and roll back the sequence on failure.
Cloudflare D1 indexes	https://developers.cloudflare.com/d1/best-practices/use-indexes/	Indexes reduce scanned rows and improve common query performance.
Cloudflare R2	https://developers.cloudflare.com/r2/	R2 is Cloudflare object storage and is the intended replacement for the user's "D2" wording.
Cloudflare Queues	https://developers.cloudflare.com/queues/	Queues are for reliable asynchronous processing; recommended only where the user is not waiting on the current request.
Cloudflare Durable Objects	https://developers.cloudflare.com/durable-objects/	Durable Objects are designed for stateful coordination such as collaborative applications.
Cloudflare Workers Rate Limiting	https://developers.cloudflare.com/workers/runtime-apis/bindings/rate-limit/	Workers Rate Limiting is suitable for fast abuse protection but is intentionally approximate, so quota accounting stays in D1.
Cloudflare Workers Observability	https://developers.cloudflare.com/workers/observability/	Workers provides logs, traces, metrics, and OpenTelemetry export.
Drizzle + D1	https://orm.drizzle.team/docs/get-started/d1-new	Drizzle documents direct Cloudflare D1 integration and migration workflow.
Better Auth Drizzle adapter	https://better-auth.com/docs/adapters/drizzle	Better Auth supports Drizzle and SQLite-backed adapters.
Tiptap + Next.js	https://tiptap.dev/docs/editor/getting-started/install/nextjs	Tiptap documents Next.js integration and client-side initialization to avoid SSR mismatch.
OpenRouter GPT-5.6 Luna	https://openrouter.ai/openai/gpt-5.6-luna-20260709	OpenRouter slug openai/gpt-5.6-luna; model supports structured outputs.
OpenRouter privacy/data collection	https://openrouter.ai/docs/guides/privacy/data-collection	Prompt/response logging is opt-in on OpenRouter; provider-side policies still need configuration/review.
OpenRouter model fallbacks	https://openrouter.ai/docs/guides/routing/model-fallbacks	OpenRouter supports provider/model fallback routing; blueprint limits MVP fallbacks to preserve consistency.

28.1 Final technical recommendation

Build this as a Cloudflare-native modular monolith.

Next.js + TypeScript + Tailwind + Tiptap in one app, deployed to Workers via vinext; D1 + Drizzle for relational state; private R2 for files and scalable version payloads; Better Auth for identity; OpenRouter + GPT-5.6 Luna behind a server-only AI adapter; deterministic validators and diff utilities around every high-trust AI flow. Add Queues and Durable Objects only when a real asynchronous or collaborative requirement appears.